

This homework is due 6 days later than usual, on *Tuesday 4/23* at 8pm. This is because some of the questions (labeled below) rely on the lectures on 4/15 and 4/17.

This homework has 7 questions, for a total of 100 points and 0 extra-credit points.

Unless otherwise stated, each question requires *clear, logically correct, and sufficient* justification to convince the reader.

For bonus/extra-credit questions, we will provide very limited guidance in office hours and on Piazza, and we do not guarantee anything about the difficulty of these questions.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up*.

(0 pts) 0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: Shikha Nukala, desigirl

(10 pts) 1. **Self assessment.**

Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't turn in the previous homework, then (1) state that you didn't turn it in, and (2) pick a problem that you think is particularly challenging from the previous homework, and explain the answer in your own words. You may reference the answer key, but your answer should be in your own words.

Solution: HW 10 Q4b

Question:

Suppose we repeatedly run our randomized algorithm until we get a cut of size at least $|E|/2$. Derive an upper bound on the expected number of attempts that are needed.

Solution:

Again we can make a random variable C as the number of attempts needed of the algorithm until there is a cut of size at least $|E|/2$. If each attempt of the algorithm is a trial, then the probability of success of getting a cut of size at least $|E|/2$ is p . Then we know from the last part that p is at least $1/2$ and $|E|/2$ is the output. Next we also know that the expected value is $1/p$ from the last part. Therefore if we put in p to the expected value we get $E[C] = 2$. Therefore the upper bound is 2 and the expected number of attempts needed until we get a cut of size at least $|E|/2$ is 2.

Choosing $\epsilon = 1/(2|E|)$ as mentioned in the part b of the problem we can substitute that in the equation of the bound being $\frac{\epsilon}{1+\epsilon}$ to get the new equation $\frac{1}{1+2|E|}$. Therefore the MaxCut algorithm will have a cut of size at least $|E|/2$ with the probability $p \geq \frac{1}{1+2|E|}$. Therefore we can see that by using a geometric distribution from the last part the expected number of attempts that are needed will be $1/p \leq 2|E| + 1$ derived from the above equation. Instead of calculating a possible value as was done in the prior solution to the problem, the problem needed a resolute bound of what the expected number of attempts needed to be for the cut size provided.

2. Short-answer potpourri.

- (5 pts) (a) Suppose that Alice's bit string (a binary number) is $x = 100000100$ and Bob's bit string is $y = 10010001$. List, with justification, all the "bad" primes p for which the fingerprinting protocol from class will *fail* to detect that $x \neq y$. (Recall that x and y are interpreted as integers written in base two, with digits written from most to least significant.)

Solution: The fingerprinting protocol follows the logic that for bit strings x and y if they differ at any position then the XOR operation will have a non-zero result while if they are equal the XOR result will be zero. Therefore the result from the XOR operation between 100000100 and 10010001 will be 110010101 . To identify the bad primes we can check the divisibility of the result on the main prime numbers. If the result is divisible, then the number is a bad prime.

For 2, the result is not divisible since the XOR result ends in 1 not 0. For 3, the sum of the digits in the result is 4, so it is not divisible. For 5, the last digit is not 0 thus is not divisible. For 7, the alternating 3 digit sums $101 - 100 + 001$ which is $2 - 1 + 1 = 2$ is not divisible by 7. For 11, the alternating digit sums $1 - 0 + 0 - 0 + 0 - 0 + 1 = 2$ which is not divisible by 11. Lastly, for 17, the alternating sum of digits which is 2 is not divisible by 17. Thus there are no bad primes for which this protocol fails to detect that $x \neq y$.

- (5 pts) (b) In class we saw that 2 is not a generator of $\mathbb{Z}_7^*$, but 3 is. State, with justification, all the other elements of $\mathbb{Z}_7^*$ that are generators of $\mathbb{Z}_7^*$.

Solution: In $\mathbb{Z}_7^*$, we have integers that are relatively prime to 7 under the modulus of 7. We know that a generator is an element that generates all other elements of the group $\mathbb{Z}_7^*$. Since we know 3 is a generator, we can test the next few integers to see if they are also generators.

For 4, we can start by seeing 4 raised to different powers. For example, 4^2 is 2, 4^3 is 1, and 4^4 is 4 (taking these values mod 7). After this we can see that 4^5 is 2, 4^6 is 1, etc. Therefore we have a cycle of more than two values, thus 4 is a generator for $\mathbb{Z}_7^*$. Next we can check 5. 5^2 is 4, 5^3 is 6, 5^4 is 2, 5^5 is 3, 5^6 is 1, 5^7 is 4 and the cycle repeats. Therefore we have 5 as a generator too. Next we can check 6. 6^2 is 1, 6^3 is 6, and 6^4 is 1. Since the cycle only repeats after 2 elements this is not a generator. The next number is 7 thus we have reached the end of the generator elements. So we have 4 and 5 as the other generators of $\mathbb{Z}_7^*$.

- (5 pts) (c) *This part relies on material from the lecture on 4/15.*

On the distant planet of Arrakis, every month has exactly 35 days (numbered from zero). The Bene Gesserit's plans are measured in *centuries*. One month, on the 0th day, they set a plan in motion that, exactly 3^{28} days later, will finally defeat the tyrannical Emperor Leto (the son of the Kwisatz Haderach, of course). Determine the day of the month when this will occur. Do not use any electronic computations, use as little by-hand calculation as you can, and show your work.

Solution: We can start by finding a different way to write 3^{28} . We know that 3^2 is 9 so we can divide 28 by 2 to get 14 and rewrite this as 9^{14} . Next we know that there are exactly 35 days per month, so we take a modulus of 9¹⁴. Thus we have $9^{14} \bmod 35$. To calculate this we can rewrite this as $81^7 \bmod 35$. From this we can take out the 81 by applying the mod operator to get $11^7 \bmod 35$. From this we can calculate as $11 * 121^3 \bmod 35$ which becomes $11 * 16^3 \bmod 35$. Next we can calculate this by writing it as $11 * 16 * 256 \bmod 35$ which becomes $11 * 16 * 11 \bmod 35$. This we can see is $16 * 121 \bmod 35$ which becomes $16 * 16 \bmod 35$ which is again $256 \bmod 35$ which is 11. Therefore at the end we can see that the defeat will occur on the 10th day of the month.

- (5 pts) (d) *This question relies on material from the lecture on 4/17.*

Pam shows Creed two pictures and claims that they are different, but Creed cannot see any difference between them. Pam wants to convince Creed that they really are different, but without revealing what the difference is. How can Creed test Pam so that if the pictures really are different, Pam will pass the test while revealing nothing more than this fact to Creed; and if the pictures are identical, Pam will fail the test with probability at least 99%? (Just describe the test; you do not need to prove any of its properties.)

Solution: Creed can conduct a blind A/B test. To outline this:
First the test provides Creed with the two pictures covered so Creed does not know

which picture is inside either. Then Creed randomly will select one of the covered pictures and look at the picture. Then Creed states whether he thinks the picture he saw is the same as the other covered picture or not without knowing if the pictures are the same or different. Then the second picture is revealed to him. If the pictures are different, then Pam passes the test if Creed said that the pictures were different. If they are identical, Pam fails because although she knows the pictures are identical, there is a high chance of 99 percent or greater that Creed believes they are different. Therefore Pam would fail the test because there isn't an actual difference. This test allows Creed to test the claim without revealing a specific difference and creates a scenario where Pam will most probably fail the test if she is false in the claim that the pictures are different.

3. Near median.

In the Quicksort algorithm, we would like to use a pivot that is “nearly” the median value of the array, because it partitions the array into two roughly equal-sized parts for recursive sorting. In this problem, we will develop a randomized algorithm for finding a near-median in *constant* time (with good probability), independent of the input size.

Let S be an array of n orderable elements, which are distinct without loss of generality. We say that $a \in S$ has *rank* $\text{rank}(a) = k$ if a is the k th smallest element in S . Given input S , the goal is to find an element having rank between $2n/5 + 1$ and $3n/5$ (inclusive). For simplicity, *we assume that n is divisible by 5* (e.g., we can “pad” the end of the array with up to four extra elements).

(5 pts) (a) Consider the simplest algorithm:

Choose an element $a \in S$ uniformly at random.

Prove that $\Pr[\text{rank}(a) \leq 2n/5] = 2/5$ and $\Pr[\text{rank}(a) > 3n/5] = 2/5$. Conclude that this algorithm is correct with probability at least $1/5$.

Solution: To prove that the algorithm is correct with probability at least $1/5$, it is necessary that the probability of selecting element a with a rank less than or equal to $2n/5$ is $2/5$ and a rank greater than $3n/5$ is also $2/5$ as per the question.

First considering the $\Pr[\text{rank}(a) \leq 2n/5]$, we know that the probability of choosing an element a less than or equal to this rank is the probability of selecting one of the $2n/5$ smallest elements out of the total of n elements. Therefore we have a simple percentage computation because a is chosen at random out of n , therefore the probability of getting one of those $2n/5$ elements out of n elements is simply $2n/5 / n$ which is $2/5$. Therefore $\Pr[\text{rank}(a) \leq 2n/5] = 2/5$.

For the $\Pr[\text{rank}(a) > 3n/5]$, we can simply subtract since we know the number of elements greater than $3n/5$ is $n - 3n/5$ which is $2n/5$. Therefore the random probability of getting one of those $2n/5$ elements selected is also $2/5$ as $2n/5 / n$ is $2/5$. Thus we also have $\Pr[\text{rank}(a) > 3n/5] = 2/5$. Since both of these hold true, we can find the total probability of selecting an element that is in between these two bounds such that $2n/5 < \text{rank}(a) \leq 3n/5$. This becomes $1 - \Pr[\text{rank}(a) \leq 2n/5] - \Pr[\text{rank}(a) > 3n/5]$

which is $1 - 2/5 - 2/5 = 1/5$. Therefore the algorithm is correct with probability at least $1/5$.

- (7 pts) (b) The previous result is not too bad, but its probability of correctness isn't so good. We would like to improve the success probability to at least $1 - \delta$, for any desired $\delta > 0$. Consider the following algorithm:

Uniformly and independently sample k elements from S *with replacement* (i.e., with repetitions allowed), for some *odd* k to be determined below. Return the median of these samples. (Because k is odd, the median is uniquely determined.)

Let X^{tiny} and X^{huge} denote the number of sampled elements whose ranks are, respectively, at most $2n/5$, and more than $3n/5$. Prove that the algorithm's output is correct if both $X^{tiny}, X^{huge} < k/2$.

Solution: Since we know there are n elements in S , n is divisible by 5, and we have to find an element in S with a rank between $2n/5+1$ and $3n/5$ inclusive, we can consider the two probabilities given in the problem.

For the probability of $X^{tiny} < k/2$, we know that X^{tiny} is the number of sampled elements which have ranks of $2n/5$ or less. Therefore like the last part, the probability of selecting these $2n/5$ elements out of n elements is $2/5$. Next, we can use linearity of expectation to find the expected value of X^{tiny} which is $k * 2/5$ which is equal to $2k/5$. From this we can see that this expected value is less than $k/2$, and by Chernoff bounds, we know that the probability of X^{tiny} straying from the expected value is small.

For $X^{huge} < k/2$, we know that X^{huge} is the number of sampled elements that are greater than $3n/5$ in rank. From the last problem, we know that this is $n - 3n/5$ elements or $2n/5$. We know that the probability of choosing $2n/5$ elements out of n is $2/5$. Therefore by linearity of expectation, the expected value is also $k*2/5$ which is $2k/5$. From this we can see that this expected value is less than $k/2$, and by Chernoff bounds, we know that the probability of X^{huge} straying from the expected value is small. Therefore we know that the algorithm's output will be correct if both X^{huge} and X^{tiny} are less than $k/2$ since their expected values are each $2k/5$. Therefore by choosing k to be accurately large, the probability of both events occurring is at least $1 - \delta$, for any desired $\delta > 0$.

- (5 pts) (c) Determine, with proof, a *constant* value of k (which may depend on δ , but not on n) so that $\Pr[X^{tiny} \geq k/2] \leq \delta/2$ and $\Pr[X^{huge} \geq k/2] \leq \delta/2$.

Hint: Set up and use an appropriate Chernoff bound.

Solution: To determine a constant k , we can say that p is the probability of each sampled element having a rank of $2n/5$ or less or greater than $3n/5$. Therefore from the last parts, we know that p is equal to $2/5$. Then we have X^{tiny} and X^{huge} as binomial random variables with parameters k and p . Since we want $\Pr[X^{tiny} \geq k/2] \leq \delta/2$ and $\Pr[X^{huge} \geq k/2] \leq \delta/2$, we can have $1 + \delta$ be $3/2$ to ensure we have $\delta/2$ as stated in

the problem. Therefore we have $\delta = 1/2$. Now if we solve the Chernoff bound for k , we have $\Pr[X^{tiny} \geq k/2] \leq \delta/2$: $e^{-\frac{(1/2)^2 * (2k/5)}{3}} \leq 1/2$. Therefore to solve this by taking the natural log of both sides, we end up with $k \geq -60 \ln(1/2)$ which is approximately equal to 41.2. Since k is a constant, we have $k = 42$ as the next smallest integer to ensure the probabilities hold.

- (5 pts) (d) Conclude that the algorithm is correct with probability at least $1 - \delta$.

Solution: For each probability, we can say that $X^{tiny} \geq k/2 \leq \delta/2$ and $X^{huge} \geq k/2 \leq \delta/2$ due to the Chernoff bound. Therefore we know that the probability that both $X^{tiny} < k/2$ and $X^{huge} < k/2$ is at least $1 - \delta$, which implies the algorithm is correct with at least that probability.

4. Diffie–Hellman.

Daphne and Julie are writing the solutions to Homework 11. Daphne has a draft of the solutions on her laptop and wants to send them confidentially to Julie. Since they have not had a private moment to choose a shared secret key together, they decide to use the Diffie-Hellman protocol to establish one over the public network.

- (5 pts) (a) Suppose they use the Diffie–Hellman protocol with the small prime $p = 19$ and generator $g = 2$ for $\mathbb{Z}_p^*$. Daphne chooses secret exponent $a = 7$, and Julie chooses $b = 11$. Derive the values that they send each other and the secret key that each one computes, to conclude that they both compute the same value.

You may use a calculator for basic arithmetic (multiplication and division), but beyond that you should use the efficient algorithms that a computer would employ. Show your work (repeated squaring etc.), but you can omit the details of modular reductions. Keep numbers small by reducing modulo p where appropriate, and use negative number where they help.

Solution: We know that we have the prime $p = 19$, the generator $g = 2$, Daphne's exponent as $a = 7$, and Julie's exponent as $b = 11$.
For Daphne, we have $2^7 \bmod 19$. Simplifying this, we have $128 \bmod 19$ which is 14.
For Julie, we have $2^{11} \bmod 19$ which is $2048 \bmod 19$ which is 15.
From these values, each sends the answer to the other.
Daphne uses the answer 15 as her new generator. Therefore we have $15^7 \bmod 19$. This is 13.
Julie calculates $14^{11} \bmod 19$. This is equal to 13.
Therefore both have computed 13 as the key using the protocol.

- (12 pts) (b) After realizing that the parameters from the previous part are much too small (since they are breakable by hand), Daphne and Julie decide to use the Diffie–Hellman protocol with a different, huge prime p and generator g of $\mathbb{Z}_p^*$. As specified by the protocol, Daphne chooses secret $a \in \mathbb{Z}_p^*$ uniformly at random, and sends $x = g^a \bmod p$ to Julie. Similarly, Julie chooses secret $b \in \mathbb{Z}_p^*$ uniformly at random, and sends $y = g^b \bmod p$ to Daphne.

Unbeknownst to them, there is a student Malcolm in the middle of their network, who is able to intercept *and undetectably replace* what they send over the network with other values of his choice. (In class we considered only an *eavesdropper* Eve who can merely read all the network traffic; this Malcolm is more powerful.)

Specifically, Malcolm chooses a secret $c \in \mathbb{Z}_p^*$ himself, and sends $z = g^c \bmod p$ to both Daphne and Julie in place of their messages to each other. That is, Daphne thinks that Malcolm's message came from Julie, and Julie thinks that Malcolm's message came from Daphne.

1. Give an expression for the key that Daphne k_D computes, and the key k_J that Julie computes, in terms of the secret exponents a, b, c and the public values p, g .

Solution: From the question, we know that Daphne's exponent is a , Julie's is b , Malcolm's is c , the prime is p , and the generator is g . Therefore from the intercepted messages, Daphne will instead receive $z = g^c \bmod p$ from Malcolm and Julie will instead receive $z = g^c \bmod p$ from Malcolm. We know that k_D is now computed as $z^a \bmod p$ and k_J is now computed as $z^b \bmod p$. To remove the z , we know that $z = g^c \bmod p$ and $\bmod p$ is part of the equation of the keys, so each key is now $k_D = (g^c)^a \bmod p$ and $k_J = (g^c)^b \bmod p$.

2. Daphne and Julie work on the homework solutions by encrypting their messages using the keys they computed in the previous part (which they believe are the same key). Malcolm knows the encryption scheme that Daphne and Julie are using. Describe how Malcolm can decrypt all of Daphne and Julie's communications without being detected, i.e., so that they believe they are communicating confidentially, and when they decrypt the ciphertexts they receive, they get exactly the messages that were intended for them.

A valid solution should show the following:

- how Malcolm can successfully decrypt all of the ciphertexts that Daphne and Julie send to each other;
- how Malcolm can avoid detection by replacing the ciphertexts that Daphne and Julie send with ones that will yield the intended messages when they decrypt (using what they believe to be their shared key).

Solution: Since Malcolm knows the encryption scheme and that Daphne and Julie are using the same key, he can decrypt and avoid detection.

First he will intercept the ciphertext that Daphne sends to Julie and that Julie sends to Daphne. Then to decrypt he will use the shared key he knows on the ciphertexts he intercepted. On the two ciphertexts C_D and C_J , Malcolm will replace them with his own ciphertexts. To generate these new ciphertexts, if M_D is the message that Daphne wants to send and M_J is the message that Julie wants to send, Malcolm encrypts the messages using the shared key then creates ciphertexts $C1_D$ and $C1_J$ that are different than C_D and C_J but yield the same messages as were intended when decrypted with the shared key. Then Malcolm sends $C1_D$ and $C1_J$ to the recipients pretending that it is from the other. Therefore when

Daphne and Juile receive the messages and decrypt them using the key, they will get the intended messages without realizing that the ciphertexts were intercepted. Therefore all of their messages can be decrypted without detection.

5. **Diffie–Hellman as public-key encryption.** *This question relies on some material from the lecture on 4/15.*

In class we said that Diffie–Hellman is an interactive protocol for two parties to establish a shared secret over a public channel, but it was not until RSA that a full public-key encryption scheme was known. However, it turns out that, by viewing the Diffie–Hellman protocol in the right way, we can actually use it for public-key encryption! (This perspective was discovered several years after RSA was proposed, and is widely used in practice now.)

Let p be a huge prime and g be a generator of $\mathbb{Z}_p^*$, both public. To generate a public key, Alice simply performs her “side” of the DH protocol: she chooses a secret exponent $a \in \mathbb{Z}_p^* = \{1, \dots, p-1\}$ uniformly at random, and publishes $x = g^a \bmod p$ as her public key.

- (6 pts) (a) Suppose that somebody (Bob, or Charlie, or Dan, etc.) wants to send Alice a confidential message m using her public key (and the other public data). Describe what the sender should do to accomplish this, and how Alice can compute the intended message from what is sent to her. (The algorithms run by Alice and the sender should be efficient.) Also give some brief intuition for why the method is plausibly secure against an eavesdropper. (You may assume that the DH protocol itself is secure, and that the sender has Alice’s genuine public key.)

Hint: Have the sender perform the other “side” of the DH protocol, and also compute and send something more. How can the message be hidden so that only Alice can compute it using her secret key?

Solution: First Bob should chose a secret exponent b such that is random so $b \in \mathbb{Z}_p^* = \{1, \dots, p-1\}$. Next Bob should compute the modulus with this exponent such that $y = g^b \bmod p$. Next with Alice’s answer, he should compute the shared key with the operation $k = x^b \bmod p$. After encrypting the message m using the key k using a symmetric encryption algorithm he can obtain the ciphertext c . Then he sends both y and c to Alice. From Alice’s side, she receives y and c . She computes the shared key $k = y^a \bmod p$. Then she decrypts c using the key k to obtain the message m . From this method we can see that only Alice who knows her exponent a can compute the key k to decrypt the ciphertext. Someone who is trying to intercept only knows x , y and g and does not know k . Therefore they cannot decrypt c to get m . Additionally, the key is generated from the discrete logarithm problem which much be solved after enough cycles in the chosen cyclic group, therefore it is relatively hard to intercept the messages.

- (5 pts) (b) In class we mentioned a security problem with the oversimplified RSA encryption scheme, as we presented it. Briefly describe the problem, and explain why the DH-style encryption scheme from the previous part does *not* suffer from this problem.

Solution: The oversimplified RSA encryption scheme directly encrypts the message using the recipient's public key. It does not have any randomness associated with it. If the same message is encrypted many times using that key, the repetition of ciphertexts can be detected and patterns can be found in the phrases and messages sent by an interceptor. With the DH-style encryption scheme, there is randomness associated with the key usage because both parties sending the messages choose secret exponents, which gives them a shared secret key k that is random or different for each encryption. Therefore even if the same message is encrypted many times with the recipient's public key, the ciphertexts generated each time will be different when the messages are sent because a secret exponent is chosen each time, which prevents patterns and similar phrases from being detected in the ciphertexts by interceptors.

6. **RSA.** *This question relies on material from the lecture on 4/15.*

Here you will run the RSA signature scheme on some small “toy” parameters. You may use a calculator for basic arithmetic (multiplication and division), but beyond that you should use the efficient algorithms that a computer would employ (where applicable), and show your work. Keep the numbers small by reducing modulo n where appropriate, and use negative numbers where they help.

Let $n = 77$ be the public modulus and $e = 7$ be the public exponent.

- (5 pts) (a) Compute the private exponent d using the extended Euclidean algorithm (which we have repeated below for convenience).

Input: integers $x \geq y \geq 0$, not both zero

Output: a triple (g, a, b) of integers where $g = \gcd(x, y)$ and $ax + by = g$

```

1: function EXTENDEDEUCLID( $x, y$ )
2:   if  $y = 0$  then
3:     return  $(x, 1, 0)$                                 ▷ Base case:  $1x + 0y = \gcd(x, y) = x$ 
4:   else
5:     Write  $x = qy + r$  for an integer  $q$ , where  $0 \leq r < y$ 
6:      $(g, a', b') \leftarrow \text{EXTENDEDEUCLID}(y, r)$ 
7:      $a \leftarrow b'$ 
8:      $b \leftarrow a' - b'q$ 
9:     return  $(g, a, b)$ 

```

Solution: Given $n = 77$ and $e = 7$:

If we select two prime numbers for p and q such that the quotient of the two numbers is 77, we have p is 11 and q is 7, we can calculate the RSA modulus $(p-1)(q-1)$ which is $mn = 10 \cdot 6 = 60$ in this case. Next, we know that we are given $e = 7$ as the public key which is greater than 1 and less than the modulus chosen. Now we have to solve for d using the equation $e \cdot d \bmod mn = 1$. To do this we input 60 for x and 7 for y . From this

we get $\text{ExtendedEuclid}(60,7)$: $60 = 7(8) + 4$. This turns into $\text{ExtendedEuclid}(7,4)$: $7 = 4(1) + 3$. This turns into $\text{ExtendedEuclid}(4,3)$: $4 = 3(1) + 1$. This turns into $\text{ExtendedEuclid}(3,1)$: $3 = 1(3) + 0$ which gives us $\text{ExtendedEuclid}(1,0)$: which is the base case which returns $(1,1,0)$. Otherwise we can solve as $60x + 7y = 1$. Therefore $60 = 7(8) + 4$ then $7 = 4(1) + 3$ then $4 = 3(1) + 1$. With back substitution we have $1 = 4 - 3(1)$ then $1 = 4 - 3(7 - 4(1))$ so $1 = 4(60 - 7(8)) - 3(7)$ which turns the last factor to $8 * 3$ or 24. Therefore we have $60 - 24$ which is $36 = d$.

- (5 pts) (b) Next, compute the correct signature for $m = 3$.

Solution: Using the private exponent d from the last part, we know that we can compute the signature for m by solving $m^d \bmod n$ which turns into $3^{36} \bmod 77$ which is 36. So the signature for $m = 3$ is 36.

- (5 pts) (c) Show how one would verify, using only the public information n, e , that the signature s you found in the previous part is correct for $m = 3$.

Solution: In order to verify the signature is correct with $n = 77$, $e = 7$, and $m = 3$. For this we need to verify that s^e is equivalent to $m \pmod{n}$. Therefore we know that $s = 3$, so we should compute $s^e \pmod{77}$. Using a calculator we have $3^7 \pmod{77}$ which gives us 36.

- (5 pts) (d) Now suppose that we had a huge (non-“toy”) RSA modulus n that was, say, 1000 bits or more in size (i.e., $n \geq 2^{1000}$). Which algorithm(s) that you used in the previous parts are *feasible* for an ordinary computer to perform for this size, and which would be *infeasible* for even a state-of-the-art supercomputer to perform? Explain your reasoning.

Solution: From the previous parts we have modular exponentiation. This is done by computing $a^b \pmod{n}$. From this we can see that a and b are larger integers and this operation can be efficient and commonly used in RSA encryption for a large RSA modulus n of 1000 bits as it often uses large numbers.

Another method is the extended euclidean algorithm. This is done by finding the modular inverse of an integer n using the extended Euclidean algorithm. This is also a feasible operation for a computer for n of 1000 bits or more because although the algorithm increases in complexity with larger sizes of n , it can still be done as is the modular exponentiation method.

Using the discrete logarithm problem would not be feasible for an n of 1000 bits or more. Since we have to find the private exponent d from the exponent e and modulus n , it has exponential time complexity because of the algorithms that are used for solving the problem so large moduli are not feasible.

The last method used was the first problem factorization method. To factor a large modulus into the prime factors with algorithms including division would have exponential time complexity and would be very inefficient on large inputs.